

WABA Group — Plateforme Lakehouse financière multi-pays

Write-up technique · Challenge Data Engineer Artefact (Banque & Assurance, Afrique de l'Ouest)

Mohamed FOFANA — octobre 2026 — dépôt : github.com/weedsterling/waba-lakehouse

1. Contexte et résultat

WestAfrica BancAssur (WABA) opère dans 8 pays (CI, SN, ML, BF, GN, TG, BJ, GH) sur 4 lignes métier (banque, assurance, mobile money, crédit) avec des données dispersées. La plateforme livrée est un **Lakehouse en architecture Lambda** qui ingère, historise, contrôle et expose ces données, du CSV d'agence jusqu'au tableau de bord réglementaire, et se déploie **en une commande sur Kubernetes** (`./scripts/k8s/deploy.sh`).

Niveau	Livré	Preuve
1 — Ingestion	MinIO + Iceberg (catalogue REST) + Spark 3.5 + Trino ; contrats de données, rejets tracés, PII masquées	MERGE idempotent : rejouer un fichier n'insère rien
2 — Batch	Airflow 3.3, médaillon bronze → silver → gold (7 KPIs), reporting BCEAO / CIMA	4 DAGs chaînés par <i>assets</i> ; exports réglementaires CSV
3 — Streaming	NiFi → Kafka (KRaft) → 2 jobs Spark Structured Streaming : fraude, AML, liquidité	Alertes dans <code>gold-*</code> en quelques secondes ; requête Lambda sans double comptage
4 — Plateforme	Kubernetes (Minikube) : 17 releases Helm, SSO Keycloak, Superset, OpenMetadata, Prometheus / Grafana / Loki	3 tableaux de bord, RLS par pays, catalogue documenté, 3 alertes <i>as code</i>

2. Architecture

```
Générateur -CSV-> MinIO raw-landing (Streamlit)
├─ Spark batch (Airflow) -> Iceberg bronze -> silver -> gold -> reporting
└─ NiFi -> Kafka raw-* -> Spark Streaming -> silver-*, gold-* (+ Iceberg rt_*)

Keycloak (SSO) -> Trino (Iceberg + Kafka) -> Superset · OpenMetadata (catalogue) · Prometheus/Grafana/Loki
```

- **Batch layer, source de vérité.** Spark lit les CSV, valide, écrit en `MERGE` dans Iceberg (partition `country_code + days(ts)`), puis Airflow enchaîne silver (dédoublonnage, conversion EUR, jointures) et gold.
- **Speed layer.** NiFi publie chaque fichier dans Kafka. Le job 1 réutilise à *l'identique* les contrats et transformations du batch (une seule définition des règles métier) ; le job 2 applique les règles de fraude (rafales sur fenêtre glissante, pays inhabituel, sinistre > 3 × prime), AML (seuils déclaratifs UEMOA / Ghana) et liquidité.
- **Serving.** Trino interroge Iceberg et Kafka dans la même requête : la vue Lambda prend le batch pour les jours calculés et le temps réel pour les suivants, sans double comptage.

3. Décisions structurantes et compromis

Décision	Alternatives	Pourquoi	Compromis accepté
Apache Iceberg , catalogue REST	Delta Lake, Hudi ; Hive Metastore	Format ouvert lu nativement par Spark et Trino, partition cachée, <i>time travel</i> ; REST sans dépendance Hadoop	Catalogue de démonstration mono-instance (PostgreSQL sur Kubernetes) ; en production : Polaris ou Lakekeeper
Idempotence à 3 niveaux (dédoublonnage du lot, <code>MERGE</code> sur clé + pays, archivage)	append + nettoyage aval	Rejouer un fichier ou un DAG ne crée jamais de doublon : prérequis d'un reporting réglementaire	Coût du <code>MERGE</code> , limité par l'élagage de partition sur <code>country_code</code>
Rejets tracés (<code>audit.rejected_records</code> , tous les motifs) + DLQ Kafka	<code>DROPMALFORMED</code>	Aucune ligne perdue silencieusement : auditabilité bancaire	Stockage supplémentaire
PII pseudonymisées à l'ingestion (IBAN masqué + SHA-256 salé)	chiffrement réversible, tokenisation	Jamais de donnée sensible en clair, hash joignable entre couches	Rotation du sel = recalcul ; le sel n'est jamais régénéré par les scripts
Tables temps réel séparées (<code>silver.rt_*</code> , <code>gold.rt_*</code>)	écrire dans les tables batch	Le batch réécrit ses partitions sans conflit d'écriture avec le flux	Une vue Lambda est nécessaire pour unifier
Append exactement-une-fois (n° de micro-lot gravé dans le snapshot Iceberg)	<code>MERGE</code> à chaque micro-lot	Un redémarrage ne perd ni ne duplique rien, sans coût de <code>MERGE</code>	Logique de reprise à maintenir côté code

Décision	Alternatives	Pourquoi	Compromis accepté
Kafka KRaft via l'opérateur Strimzi	ZooKeeper, chart Bitnami	Plus de ZooKeeper ; topics déclarés en <code>KafkaTopic</code> (GitOps)	Strimzi 0.45 impose Kubernetes ≤ 1.32 (version épinglée)
Spark Operator (SparkApplication) appelé par Airflow	spark-submit depuis le scheduler	Un seul modèle de job pour Airflow, les flux permanents et les relances manuelles	Opérateur supplémentaire à exploiter
Helmfile , un release par composant et par domaine	<i>umbrella chart</i> unique	Déploiement ordonné (<code>needs</code>), mise à jour ciblée d'un composant	Plus de fichiers qu'un chart unique

4. Qualité, sécurité et gouvernance

- **Qualité des données.** Contrats déclaratifs (`spark/waba_spark/schemas.py`) partagés par le batch et le streaming ; contrôles devise / pays / montants ; métriques de qualité par lot (`audit.dq_metrics`) ; journal d'ingestion par fichier (ETag, lignes lues / valides / rejetées).
- **Secrets.** Aucun secret dans le dépôt : `.env` (hors Git) généré par `gen-secrets.sh`, qui ne modifie jamais une valeur existante ; Secrets Kubernetes créés par `bootstrap.sh` ; compte de service MinIO à privilèges réduits.
- **SSO et contrôle d'accès** (Keycloak 26, realm `as code` resynchronisé à chaque déploiement). Quatre rôles métier (`group_admin`, `country_analyst`, `compliance_officer`, `viewer`) appliqués à deux endroits : Superset (rôles recalculés à chaque connexion, **RLS par pays** avec refus par défaut, tableau de bord réglementaire seul pour la conformité) et Trino (filtre de lignes par pays, **colonnes IBAN refusées** aux analystes, couche silver interdite au `viewer`). Vérifié utilisateur par utilisateur.
- **Catalogue** (OpenMetadata 1.12, démarré à la demande pour libérer 3 Go). Déclaré dans `openmetadata/catalog.py` : découverte Trino, 7 tables Gold documentées avec propriétaire et tags BCEAO / CIMA / AML, colonnes personnelles marquées `PII.Sensitive`, **lineage de bout en bout** (conteneur MinIO → bronze → silver → gold → reporting), chaque arête portant le DAG producteur.

5. Plateforme Kubernetes et observabilité

- **Déploiement.** 5 namespaces par domaine (`ingestion`, `processing`, `serving`, `governance`, `monitoring`), sondes sur chaque composant, Ingress `*.waba.local`, images préchargées dans le nœud, versions épinglées. Les scripts protègent les données : le cluster n'est jamais supprimé implicitement.
- **Supervision.** kube-prometheus-stack (Prometheus, Grafana 13, kube-state-metrics), Loki monolithique et Grafana Alloy (Promtail étant en fin de vie), droits réduits à la lecture des pods. Tableau de bord « WABA — Supervision » généré par code : état des flux, débit et lag Kafka, rapport réglementaire, mémoire par domaine, journaux en erreur.
- **Les 3 alertes** sont provisionnées depuis le dépôt dans Grafana :

Alerte	Source	Règle
Job fraude en erreur > 5 min	kube-state-metrics (état de la SparkApplication)	<code>stream-silver-gold</code> hors <code>RUNNING</code> pendant 5 min, ou supprimé
Lag consumer AML > 5 000	Kafka Exporter (Strimzi)	lag du groupe <code>waba-spark-rules</code> , la requête qui produit <code>gold-aml-events</code>
<code>dag_regulatory_report</code> en échec à 06h00 UTC	base Airflow, compte <code>grafana_ro</code> (SELECT sur <code>dag_run</code> seul)	après 06h00 UTC : aucun succès du jour, ou dernière exécution en échec

Point d'architecture : Spark Structured Streaming ne s'inscrit dans aucun groupe Kafka, sa progression vit dans le checkpoint. Un *listener* publie donc après chaque micro-lot les offsets traités dans `waba-spark-<requête>` : le lag devient mesurable par les outils standard et **continue de croître si le job s'arrête**, ce qu'une métrique émise par le job lui-même ne montrerait pas. La reprise reste pilotée par le checkpoint.

6. Ingénierie et vérification

- **≈ 140 tests pytest** (106 fonctions, dont des cas paramétrés) : transformations Spark réelles (silver, gold, fraude, streaming), contrats, DAGs, et tests de **cohérence entre sources** qui empêchent une divergence silencieuse : rôles Keycloak ↔ Superset ↔ Trino, alertes ↔ noms des jobs / requêtes / DAG, tableau de bord régénéré ↔ JSON versionné, versions des charts épinglées.
- **Tout est déclaratif et versionné** : flux NiFi, realm Keycloak, tableaux de bord Superset, catalogue OpenMetadata, alertes et tableau de bord Grafana. Un `helmfile sync` ramène la plateforme à l'état de Git.
- **Correctifs définitifs plutôt que manuels** : chaque incident rencontré a été corrigé à la source et couvert par un test de non-régression (exemples : Elasticsearch 9 exigé par OpenMetadata 1.12, données ES isolées par

version majeure, authentification Trino limitée à l'interface web pour ne pas bloquer les appels internes, relance automatique des flux Spark quand leur code change).

7. Limites connues et prochaines étapes

Limite	Prochaine étape
Un seul nœud Minikube (≈ 22 Go) : pas de haute disponibilité, OpenMetadata démarré à la demande	Cluster multi-nœuds, réplication Kafka ≥ 3, PostgreSQL managé
Catalogue Iceberg et bases de démonstration mono-instance	Polaris / Lakekeeper adossé à PostgreSQL HA
Maintenance Iceberg non planifiée	DAG de compaction, <code>expire_snapshots</code> , <code>remove_orphan_files</code>
Alerte lag AML : publication des offsets validée en local, pas encore observée dans le cluster	Exercice <code>alert-drill.sh</code> sous charge du générateur
Alertes visibles dans Grafana, sans canal de notification	Point de contact Teams / e-mail (SMTP) et astreinte
Secrets Kubernetes créés depuis <code>.env</code>	Sealed Secrets ou External Secrets (Vault)
Hypothèse de l'énoncé : Guinée en XOF (en réalité GNF, hors UEMOA)	Paramètre déjà centralisé (<code>CURRENCY_MAP</code>)

8. Utilisation de l'IA

J'ai utilisé Claude (Anthropic) comme binôme d'ingénierie, sur tout le projet : proposition d'architecture et de compromis, écriture de code et de manifestes, diagnostic d'incidents. Le cadre était fixé dès le départ : correctifs définitifs et testés plutôt que manipulations manuelles, aucun secret dans le code, sel PII jamais régénéré.

La vérification n'a jamais reposé sur l'IA :

- chaque livraison était **exécutée sur ma VM** (Ubuntu 24.04, Minikube), puis validée sur des preuves (sortie de commande, capture d'interface, requête Trino), utilisateur par utilisateur pour la sécurité ;
- chaque incident a été **diagnostiqué sur des faits** (journaux, événements Kubernetes, code source des outils) avant correction, puis couvert par un test ;
- les points non prouvés sont **déclarés comme tels** dans ce document (section 7) plutôt que présentés comme acquis.

L'IA a nettement accéléré l'écriture et le diagnostic. Les choix d'architecture, la validation et la responsabilité du résultat restent les miens.